

websites make is displaying too much information to the end user if an error occurs; sometimes, the whole PHP runtime error is displayed. This is an annoyance to the user and an opportunity for an attacker. You should always turn off 'error display' during production.

PHP comes with two config files – *php.ini-development* and *php.ini-production*. While you are developing the application locally, or in the test phase, use the development file as your default *php.ini* which is configured to log and display all types of errors.

As soon as you deploy your website live, you should start using the production that will have the 'error display' turned off and other typical production configurations set by default.

Don't store sensitive data in code but use environment variables: When we need to connect to a database, we have to let the API know its credentials. Similarly, API keys have to be passed to the language to access certain services. Often, programmers hard code these values inside the code, and this code is pushed to the version control server. The problem with this is that all the developers now have access to the database server credentials, which is inherently a bad idea.

To avoid this, we can use environment variables provided by Apache. You can achieve this by editing the virtual host configuration file for your website, which is usually available at */etc/apache2/sites-available/* with a *.conf* extension.

```
<VirtualHost *:80>
...
SetEnv DB_USER webadmin
SetEnv DB_PASS aC0mpl3XPa$sword
...
```

```
</VirtualHost>
```

You can add variables for database access in this manner. After restarting the server once, you can access them with PHP later on using the following command:

```
getenv('DB_USER')
```

This will ensure that you do not expose sensitive data like database credentials in your production/development code and, instead, have it stored in the Apache configuration which can only be accessed by a user with higher permissions (the sysadmin).

You can also use other *.env* files to store your database credentials with a popular library: <https://github.com/vlucas/phpdotenv>. This will allow you to automatically import the variables stored in *.env* files to the *getenv()* or *\$_ENV* variables in PHP.



Warning: Never allow your sensitive data to be stored in code or anywhere in the source code tree, including *.env* files!

This, in no way, is all you can do to keep your LAMP server secure. There are plenty of resources online that can help you get started with this. One such thread is at <https://serverfault.com/questions/212269/tips-for-securing-a-lamp-server>. **END** 🐧

 By: Dedipyaman Das

The author is a software developer and open source enthusiast. You can contact him at 2d@twodee.me. URL: <https://www.twodee.me>.

OSFY Magazine Attractions During 2019-20

MONTH	THEME
March 2019	Mobile/Web App Development and Optimisation
April 2019	Cybersecurity, Open Source Firewall, Network Security and Monitoring
May 2019	AI, Deep Learning and Machine Learning
June 2019	DevOps Special
July 2019	Blockchain and Open Source
August 2019	Database Management and Optimisation
September 2019	Open Source and IoT
October 2019	Cloud Special: BigData, Hadoop, PaaS, SaaS, IaaS and Cloud
November 2019	Open Source on Windows
December 2019	Open Source Programming (Languages and Tools)
January 2020	Data Security, Storage and Backup
February 2020	Best in the World of Open Source (Tools and Services)